

HDOC-28

1) Aim:

To write a C program that accepts valid row and column dimensions, creates a variable-length 2D array, reads elements column by column, and displays them in row-major order using a single loop and linear index.

Task1: Algorithm:

1. Start.
2. Declare integers rows, cols, i, j, and index.
3. Read the number of rows.
4. If rows ≤ 0 , display an error and ask for rows again.
5. Read the number of columns.
6. If cols ≤ 0 , display an error and ask for columns again.
7. Declare a variable-length array `a[rows][cols]`.
8. Read elements column by column:
 - Outer loop: $j = 0$ to $cols - 1$
 - Inner loop: $i = 0$ to $rows - 1$
 - Read `a[i][j]`.
9. Use one loop from $index = 0$ to $rows * cols - 1$.
10. Convert the linear index into:
 - $Row = index / cols$
 - $Column = index \% cols$
11. Print `a[index / cols][index \% cols]`.
12. Stop.

Task2: Test case table:

Test Case	Rows Input	Columns Input	Array Elements Entered Column-wise	Expected Output in Row-Major Order	Purpose
1	2	3	10, 40, 20, 50, 30, 60	10 20 30 40 50 60	Normal case
2	3	2	1, 3, 5, 2, 4, 6	1 2 3 4 5 6	Normal rectangular array
3	0, then 2	3	1, 4, 2, 5, 3, 6	Error for rows: Rows must be positive. Enter again. Then: 1 2 3 4 5 6	Invalid row input
4	2	-2, then 2	10, 30, 20, 40	Error for columns: Columns must be positive. Enter again. Then: 10 20 30 40	Invalid column input
5	1	1	25	25	Boundary case: smallest valid array

Task3: C Program:

```
#include <stdio.h>
int main()
{
    int rows, cols;
    int i, j, index;
    /* Read and validate number of rows */
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if(rows <= 0)
        {
            printf("Rows must be positive. Enter again.\n");
        }
    } while(rows <= 0);
```

```

/* Read and validate number of columns */
do
{
    printf("Enter number of columns: ");
    scanf("%d", &cols);
    if(cols <= 0)
    {
        printf("Columns must be positive. Enter again.\n");
    }
} while(cols <= 0);
/* Variable Length Array */
int a[rows][cols];
/* Input column by column */
printf("\nEnter array elements column by column:\n");
for(j = 0; j < cols; j++)
{
    for(i = 0; i < rows; i++)
    {
        printf("Enter a[%d][%d]: ", i, j);
        scanf("%d", &a[i][j]);
    }
}

/* Display in row-major order using one loop */
printf("\nElements in row-major order:\n");
for(index = 0; index < rows * cols; index++)
{
    printf("%d ", a[index / cols][index % cols]);
    if((index + 1) % cols == 0)
    {
        printf("\n");
    }
}
return 0;
}

```

Task4: Output:

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
```

```
Enter number of rows: 2
Enter number of columns: 3
```

```
Enter array elements column by column:
Enter a[0][0]: 10
Enter a[1][0]: 40
Enter a[0][1]: 20
Enter a[1][1]: 50
Enter a[0][2]: 30
Enter a[1][2]: 60
```

```
Elements in row-major order:
102030
405060
```

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
```

```
Enter number of rows: 3
Enter number of columns: 2
```

```
Enter array elements column by column:
Enter a[0][0]: 1
Enter a[1][0]: 3
Enter a[2][0]: 5
Enter a[0][1]: 2
Enter a[1][1]: 4
Enter a[2][1]: 6
```

```
Elements in row-major order:
12
34
56
```

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
```

```
Enter number of rows: 0
Rows must be positive. Enter again.
Enter number of rows: 2
Enter number of columns: 3
```

```
Enter array elements column by column:
Enter a[0][0]: 1
Enter a[1][0]: 4
Enter a[0][1]: 2
Enter a[1][1]: 5
Enter a[0][2]: 3
Enter a[1][2]: 6
```

```
Elements in row-major order:
123
456
```

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
```

```
Enter number of rows: 2
Enter number of columns: -2
Columns must be positive. Enter again.
Enter number of columns: 2
```

```
Enter array elements column by column:
Enter a[0][0]: 10
Enter a[1][0]: 30
Enter a[0][1]: 20
Enter a[1][1]: 40
```

```
Elements in row-major order:
1020
3040
```

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
```

```
Enter number of rows: 1
Enter number of columns: 1
```

```
Enter array elements column by column:
Enter a[0][0]: 25
```

```
Elements in row-major order:
25
```

2) Aim:

To write a C program that reads elements into a variable-length 2D array, copies the elements into a 1D array in **row-major order**, and displays both arrays.

Task1: Algorithm:

1. Start.
2. Declare rows, cols, i, j, and k.
3. Read the number of rows and columns.
4. Validate that both are positive.
5. Declare the variable-length 2D array a[rows][cols].
6. Declare the 1D array b[rows * cols].
7. Set k = 0.
8. Read the elements of the 2D array using nested loops.
9. Traverse the 2D array in row-major order.
10. Copy each element using $b[k] = a[i][j]$.
11. Increment k.
12. Display the 2D array.
13. Display the 1D array.
14. Stop.

Task2: Test case table:

Test Case	Rows	Columns	Elements Entered in 2D Array	Expected 2D Array	Expected 1D Array	Purpose
1	2	3	10 20 30 40 50 60	10 20 30 40 50 60	10 20 30 40 50 60	Normal case
2	3	2	1 2 3 4 5 6	1 2 3 4 5 6	1 2 3 4 5 6	Rectangular array
3	1	1	25	25	25	Boundary case: smallest valid array
4	0, then 2	2	5 10 15 20	5 10 15 20	5 10 15 20	Invalid rows followed by valid input
5	2	3	-5 0 10 -15 20 25	-5 0 10 -15 20 25	-5 0 10 - 15 20 25	Tests negative, zero and positive elements

Task3: C program:

```
#include <stdio.h>
int main()
{
    int rows, cols;
    int i, j, k;
    /* Input and validation of rows */
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if(rows <= 0)
        {
            printf("Rows must be positive. Enter again.\n");
        }
    } while(rows <= 0);
    /* Input and validation of columns */
    do
    {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
        if(cols <= 0)
        {
```

```

        printf("Columns must be positive. Enter again.\n");
    }
} while(cols <= 0);
/* Variable Length Arrays */
int a[rows][cols];
int b[rows * cols];
/* Read 2D array */
printf("\nEnter elements of the 2D array:\n");
for(i = 0; i < rows; i++)
{
    for(j = 0; j < cols; j++)
    {
        printf("Enter a[%d][%d]: ", i, j);
        scanf("%d", &a[i][j]);
    }
}
/* Copy 2D array into 1D array */
k = 0;
for(i = 0; i < rows; i++)
{
    for(j = 0; j < cols; j++)
    {
        b[k] = a[i][j];
        k++;
    }
}
/* Display 2D array */
printf("\n2D Array:\n");
for(i = 0; i < rows; i++)
{
    for(j = 0; j < cols; j++)
    {
        printf("%d\t", a[i][j]);
    }
    printf("\n");
}
/* Display 1D array */
printf("\n1D Array in row-major order:\n");
for(k = 0; k < rows * cols; k++)
{
    printf("%d ", b[k]);
}
printf("\n");
return 0;
}

```

Task4: Output:

```
(kali㉿kali)-[~/hdoc28.c]
$ cc ques2.c

(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
Enter number of rows: 2
Enter number of columns: 3

Enter elements of the 2D array:
10
20
30
40
50
60

2D Array:
10      30      50
20      40      60

1D Array in row-major order:
103050204060

(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
Enter number of rows: 3
Enter number of columns: 2

Enter elements of the 2D array:
1
2
3
4
5
6

2D Array:
1      4
2      5
3      6

1D Array in row-major order:
142536

(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
Enter number of rows: 1
Enter number of columns: 1

Enter elements of the 2D array:
25

2D Array:
25

1D Array in row-major order:
25
```

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
Enter number of rows: 0
Rows must be positive. Enter again
Enter number of rows: 2
Enter number of columns: 2
```

Enter elements of the 2D array:

5
10
15
20

2D Array:

5	15
10	20

1D Array in row-major order:

5151020

```
(kali㉿kali)-[~/hdoc28.c]
$ ./a.out
```

Enter number of rows: 2
Enter number of columns: 3

Enter elements of the 2D array:

-5
0
10
-15
20
25

2D Array:

-5	10	20
0	-15	25

1D Array in row-major order:

-510200-1525